

Лабораторная работа №1

Вычисление числа π с использованием OpenMP

Поздняков Илья, группа 11-401

1. Описание работы

Реализованы последовательная и параллельная версии вычисления числа π по ряду Лейбница на языке C; параллельная версия использует директиву `#pragma omp parallel for reduction(+:sum)`, время измерялось функцией `omp_get_wtime()`. Ускорение вычислено как $S = T_1/T_p$, эффективность — как $E = S/p$. Каждый замер выполнен 5 раз, в таблицах приведена медиана. Стенд: Intel Xeon 2.8 ГГц, 2 ядра (2 логических потока, без SMT), GCC 13.3, сборка `gcc -fopenmp pi.c`.

2. Результаты

Таблица 1. Влияние числа потоков ($n = 10^9$, $T_{\text{пол}} = 3.130$ с)

Потоки	Время, с	Ускорение	Эффективность
1	3.103	1.01	101 %
2	1.637	1.97	99 %
4	1.609	1.94	49 %
8	1.514	2.07	26 %
16	1.567	2.01	13 %

Таблица 2. Влияние размера задачи (8 потоков)

n	Последовательно, с	OpenMP (8 потоков), с	Ускорение
10^7	0.035	0.018	1.66
10^8	0.308	0.158	1.93
10^9	2.993	1.528	1.96
10^{10}	30.625	15.451	1.98

3. Вывод

При переходе с 1 на 2 потока время выполнения сократилось почти вдвое (ускорение ≈ 2 , эффективность около 100 %), поскольку члены ряда независимы, а синхронизация нужна только при объединении локальных сумм в reduction. При дальнейшем увеличении числа потоков до 4, 8 и 16 время почти не изменилось, то есть ускорение перестало расти уже на 2 потоках — это число физических ядер машины. Потоки сверх числа ядер не получают новых вычислительных ресурсов, а делят те же ядра, поэтому эффективность падает до ~ 13 % на 16 потоках; значения ускорения чуть выше 2 объясняются разбросом замеров. Последовательная часть программы ничтожно мала, поэтому по закону Амдала предел здесь задаёт именно число ядер; на процессоре с большим числом ядер ускорение росло бы дальше. Во втором эксперименте время обеих версий растёт линейно с n — примерно в 10 раз на каждом шаге. Ускорение растёт вместе с размером задачи: при $n = 10^7$ (около 30 мс) оно составило лишь 1.66, а при $n = 10^{10}$ — 1.98, то есть почти достигло предела для двух ядер. Чем больше размер задачи, тем меньше доля накладных расходов на создание и синхронизацию потоков (модель fork-join) и тем ближе ускорение к максимально возможному, поэтому OpenMP наиболее эффективен на больших объёмах вычислений.